Github Actions

GitHub Actions is a continuous integration and continuous delivery (CI/CD) platform that allows you to automate your build, test, and deployment pipeline. You can create workflows that build and test every pull request to your repository, or deploy merged pull requests to production.

GitHub Actions goes beyond just DevOps and lets you run workflows when other events happen in your repository. For example, you can run a workflow to automatically add the appropriate labels whenever someone creates a new issue in your repository.

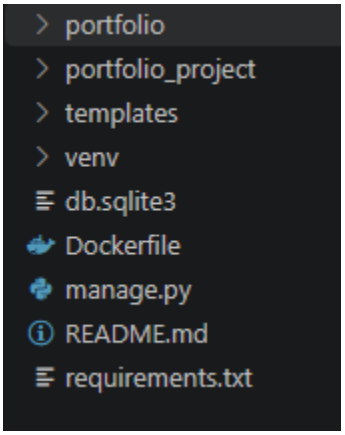
GitHub provides Linux, Windows, and macOS virtual machines to run your workflows, or you can host your own self-hosted runners in your own data center or cloud infrastructure.

For this tutorial, we assume you are familiar with Docker and Github through previous tutorials, we also assume your codebase is pushed to Github and is public. In case the required tools are not downloaded:

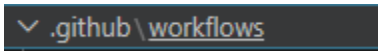
Download Docker here: https://www.docker.com/products/docker-desktop/
Download Git here: https://git-scm.com/downloads

Part 1: Setting up the ci.yml file

This is what your current codebase should look like AND it should be pushed to github as per the previous tutorial.



Create a new folder “workflows” inside the “.github” folder. If “.github” doesn’t exist, create it first.

1. You can create the folders manually or run:
   mkdir -p .github/workflows or mkdir .github\workflows
   

2. Inside the “workflows” folder, create the file “ci.yml” and paste the code below:

```
name: CI

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  test:
    runs-on: ubuntu-24.04
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v5
```

```
- uses: actions/setup-python@v6
  with:
    python-version: '3.11'

- name: Install dependencies
  run: pip install -r requirements.txt

- name: Django system check
  run: python manage.py check

- name: Log in to GitHub Container Registry
  uses: docker/login-action@v4
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}

- name: Build Docker image
  run: docker build -t ghcr.io/your-username/your-repo-name:latest .

- name: Push Docker image
  run: docker push ghcr.io/your-username/your-repo-name:latest
```

Replace < your-username/your-repo-name:latest> with your actual github username and the name you gave your repository; here ***all must be lowercase*** (even if uppercase in GitHub) and don't forget the "." at the end of the build docker image step.

File explanation:

This file defines your pipeline. GitHub automatically looks inside the .github/workflows/ folder, finds any file there, and runs it on GitHub's own servers whenever the conditions you specify are met. That means your pipeline lives in your repository, version-controlled alongside your code. Nothing to install, no server to maintain.

The important parts:

- **on:** defines when the pipeline runs. push with branches: [main] means it runs automatically on every push to the main branch. workflow_dispatch adds a "Run workflow" button so you can also trigger it manually from the Actions tab.
- **runs-on:** defines where it runs. GitHub gives you a fresh virtual machine for every run — it is created, does the work, and is destroyed. Nothing carries over between runs, so your pipeline always starts from a clean state.
- **steps:** is the list of things the pipeline does, executed top to bottom in order. If one step fails, everything after it is skipped and the run is marked as failed.
- **uses:** pulls in a prewritten action from GitHub's marketplace. For example, actions/checkout downloads your code onto the machine — without it, the machine is empty. The @v5 at the end pins the version so your pipeline does not break when a new one is released.
- **run:** executes a shell command, exactly as you would type it in a terminal. The machine runs Linux, so Linux commands are used here regardless of your own operating system.
- **permissions:** controls what the run is allowed to do. By default a workflow can only read your repository; packages: write grants it permission to publish your Docker image.
- **${{ }}** marks values that GitHub fills in automatically at runtime. github.actor becomes the username of whoever triggered the run, and secrets.GITHUB_TOKEN is a temporary password GitHub creates for that run and destroys when it finishes. You never create these or paste them anywhere.

Part 2: Push the changes to github and ensure the action fires

1. Push the changes to github:

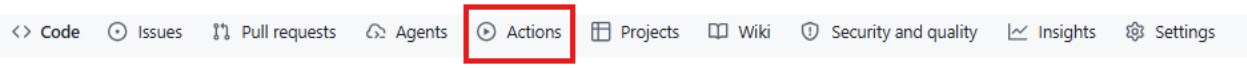
```
git add .github

git commit -m "Add GitHub Actions CI workflow"

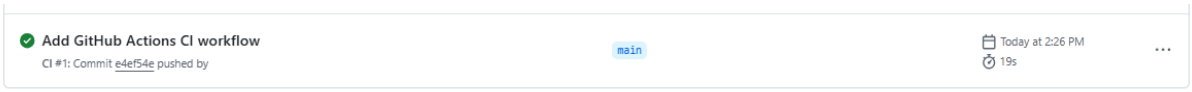
git push origin main
```

Remember that you are free to tag your commit, for the sake of this push we will call it “Add Github Actions CI workflow” but you can name it anything you want.

2. Once the push is done, go to the “actions tab” in github (top of the page):

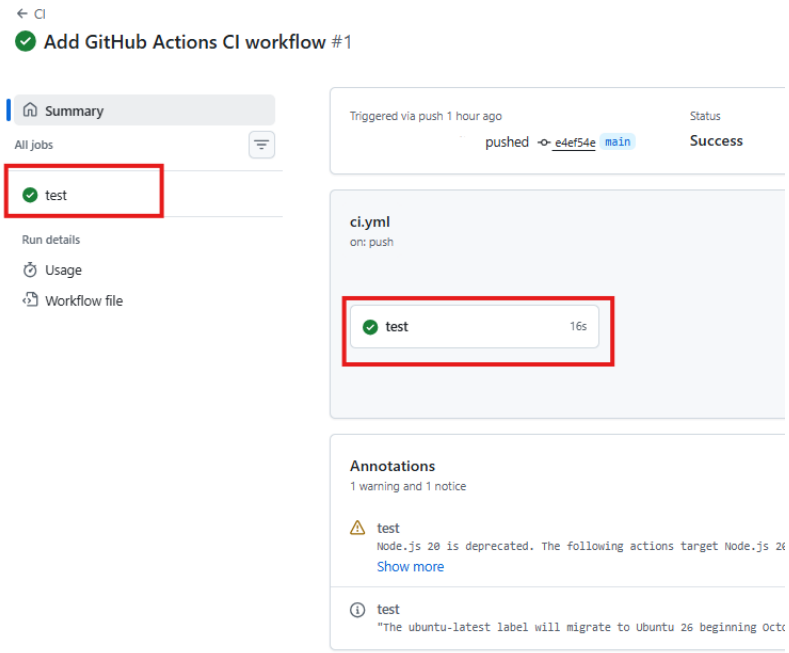


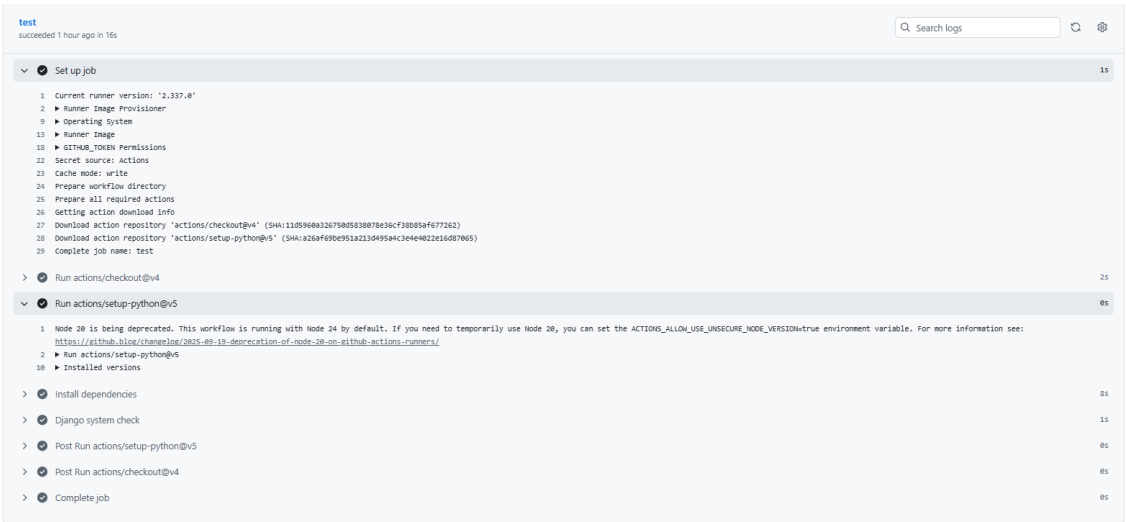
You should see the action ran successfully:



If it's still brown (loading), give it some time.

3. You can check the logs of your action by clicking on it (the Add GitHub Actions CI workflow) then clicking on "test" (note that you must be logged into github to see the logs)





Part 3: Test your CI pipeline

1. Make any change to your code, for example in “templates/home.html” then save your changes and push to github:

   ```
   git add .
   git commit -m "Testing CI"

   git push
   ```

2. Wait for your action to complete, then in cmd run:

   ```
   docker pull ghcr.io/your-username/your-repo-name:latest
   docker run -d -p 8000:8000 ghcr.io/your-username/your-repo-name:latest
   ```

   Again, with your username and repository name all lowercase.

   Note that for this step docker desktop must be running.

3. Go to localhost:8000, you should see your changes reflected.

Note:
If you want to test again, follow the testing steps but before step 2 run:

```
docker stop <container-id>
```

```
docker rm <container-id>
```

To stop and remove your current running image.

To find the container id run : docker ps



Part 4: Continuous Deployment locally

So far, our pipeline checks the Django project, builds a Docker image, and publishes it to GitHub Container Registry. We still pull and run the image manually.

We will install a GitHub Actions **self-hosted runner** on our computer. This lets GitHub Actions execute deployment commands locally and update our running Docker container automatically after CI succeeds.

For deployment to work, our computer, Docker Desktop, and the runner must be running. We will access the website at http://localhost:8000.

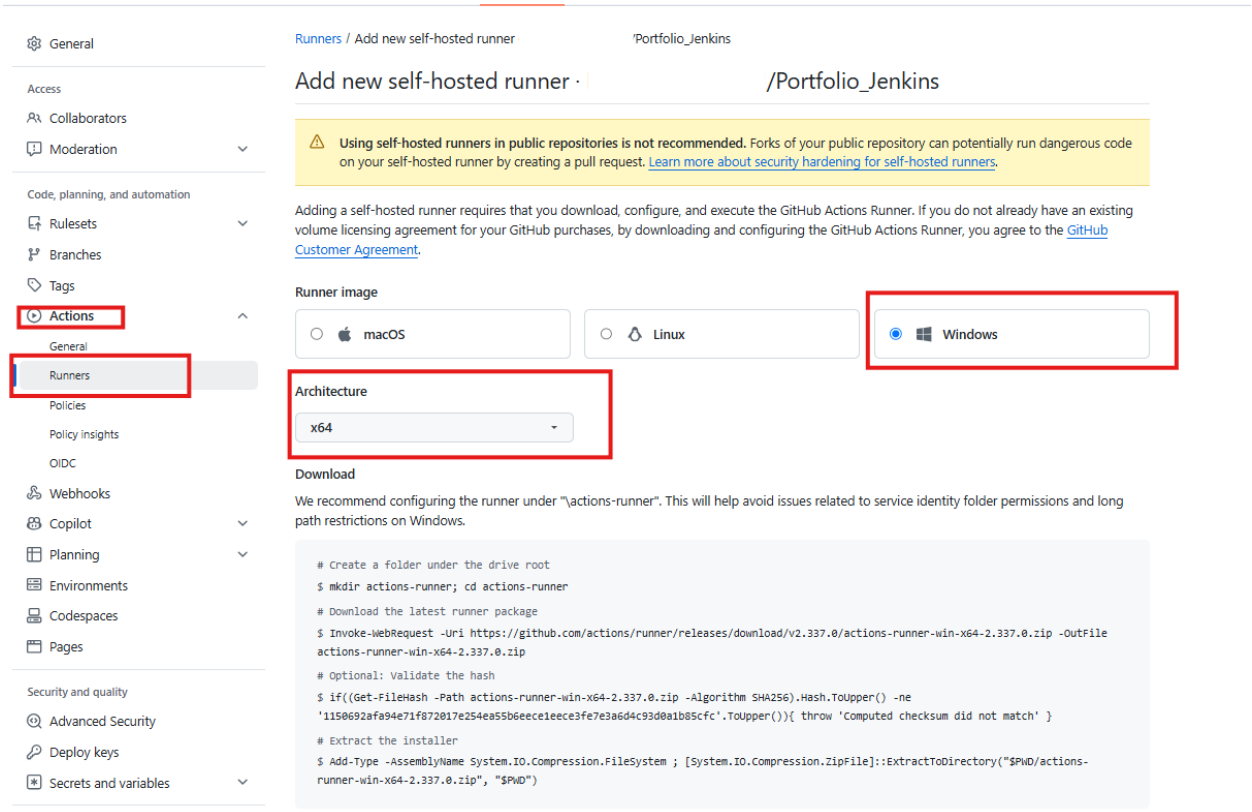
1. Verify that Docker Desktop is ready
   In cmd run: docker info --format "Docker server: {{.ServerVersion}} | Container OS: {{.OSType}}"

   ```
   C:\Users\ \Desktop\Portfolio_Jenkins>docker info --format "Docker server: {{.ServerVersion}} | Container OS: {{.OS
   Type}}"
   Docker server: 28.0.1 | Container OS: linux
   ```

   The version may differ. Make sure **Container OS** is **linux**, because our CI pipeline builds a Linux image.
   If the command cannot connect to Docker, make sure Docker Desktop is running. If it shows windows, switch Docker Desktop to Linux containers and run the command again.

2. Open the self-hosted runner setup page
   a. Open your repository on GitHub.
   b. Click Settings.
   c. In the left sidebar, expand Actions, then click Runners.
   d. Click New self-hosted runner.
   e. Select Windows (MAC) as the runner image and x64 as the architecture.

   The runner will execute deployment commands on our Windows computer.



c

3. Open Powershell (command prompt doesn't work) and create the runner folder ***OUTSIDE*** your project folder:
   New-Item -ItemType Directory -Path C:\actions-runner
   Set-Location C:\actions-runner
   For Mac, Open terminal and run:
   mkdir -p ~/actions-runner
   cd ~/actions-runner

4. From the GitHub's Download section (in the screenshot above) , copy and execute the commands for:
   a. Download the latest runner package (this might take a few minutes)
   b. Validate the hash
   c. Extract the installer

   Don't include the '$' at the beginning of the command

```
PS C:\actions-runner> Invoke-WebRequest -Uri https://github.com/actions/runner/releases/download/v2.337.0/actions-runner
-win-x64-2.337.0.zip -OutFile actions-runner-win-x64-2.337.0.zip
PS C:\actions-runner> if((Get-FileHash -Path actions-runner-win-x64-2.337.0.zip -Algorithm SHA256).Hash.ToUpper() -ne '1
150692afa94e71f872017e254ea55b6eece1eece3fe7e3a6d4c93d0a1b85cfc'.ToUpper()){ throw 'Computed checksum did not match' }
PS C:\actions-runner> Add-Type -AssemblyName System.IO.Compression.FileSystem ; [System.IO.Compression.ZipFile]::Extract
ToDirectory("$PWD/actions-runner-win-x64-2.337.0.zip", "$PWD")
PS C:\actions-runner>
```

5. To verify correctness run: Get-Item .\config.cmd, .\run.cmd OR FOR MAC ls -l config.sh run.sh

```
PS C:\actions-runner> Get-Item .\config.cmd, .\run.cmd

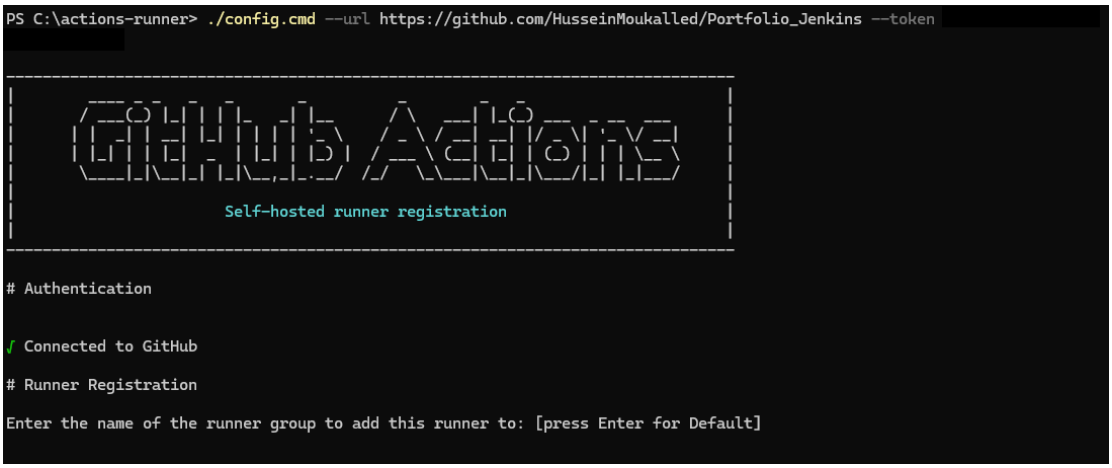

    Directory: C:\actions-runner


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         8/26/2026   2:29 PM           1225 config.cmd
-a----         8/26/2026   2:29 PM           1297 run.cmd


PS C:\actions-runner>
```

Part 5: Configure the self-hosted runner

1. On GitHub's runner setup page, find Configure. Copy its configuration command and run it from your runner folder.



2. You will be asked a set of questions:

   - Keep default for runner group
   - Set name of runner to "portfolio-local"
   - Add 'portfolio-deploy" as additional label
   - Keep default for work folder

- Choose N for running the service

```
# Authentication


√ Connected to GitHub

# Runner Registration

Enter the name of the runner group to add this runner to: [press Enter for Default]

Enter the name of runner: [press Enter for DESKTOP-95AFOF3] portfolio-local

This runner will have the following labels: 'self-hosted', 'Windows', 'X64'
Enter any additional labels (ex. label-1,label-2): [press Enter to skip] portfolio-deploy

√ Runner successfully added

# Runner settings

Enter name of work folder: [press Enter for _work]

√ Settings Saved.

Would you like to run the runner as service? (Y/N) [press Enter for N] N
PS C:\actions-runner>
```

Part 6: Start the self-hosted runner

1. Run .\run.cmd OR FOR MAC ./run.sh

```
PS C:\actions-runner> .\run.cmd
        1 file(s) copied.

√ Connected to GitHub

Current runner version: '2.337.0'
2026-10-11 16:28:08Z: Listening for Jobs
```

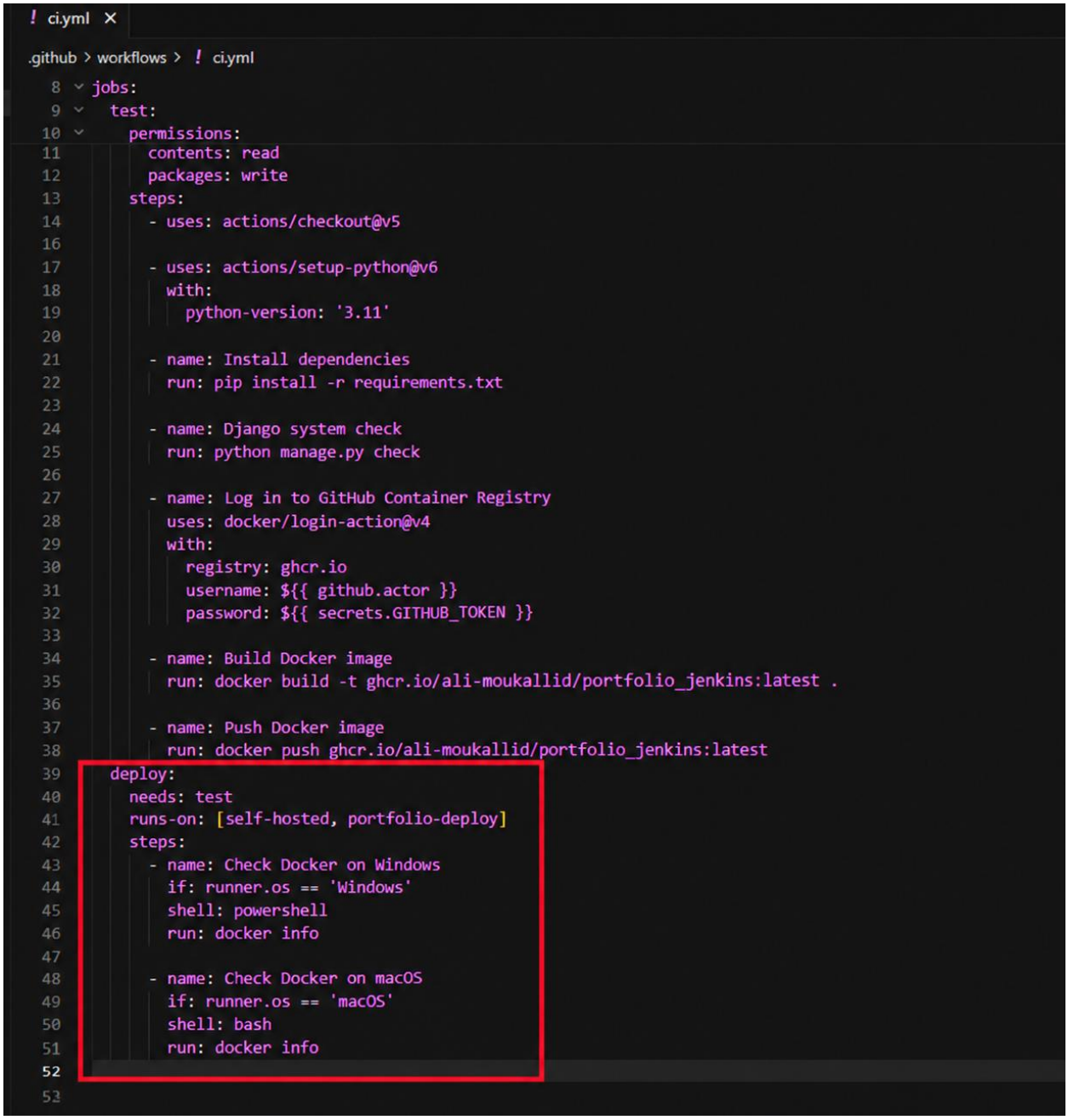
Note that you must always have the runner active for the CD to work.

2. Verify that GitHub Actions can access local Docker
   Add the code below at the end of the ".github/workflows/ci.yml". Make sure the indentations are correct as per the screenshot below.

```
deploy:
  needs: test
  runs-on: [self-hosted, portfolio-deploy]
  steps:
    - name: Check Docker on Windows
      if: runner.os == 'Windows'
      shell: powershell -NoProfile -ExecutionPolicy Bypass -File {0}
      run: docker info

    - name: Check Docker on macOS
      if: runner.os == 'macOS'
      shell: bash
```

run: docker info

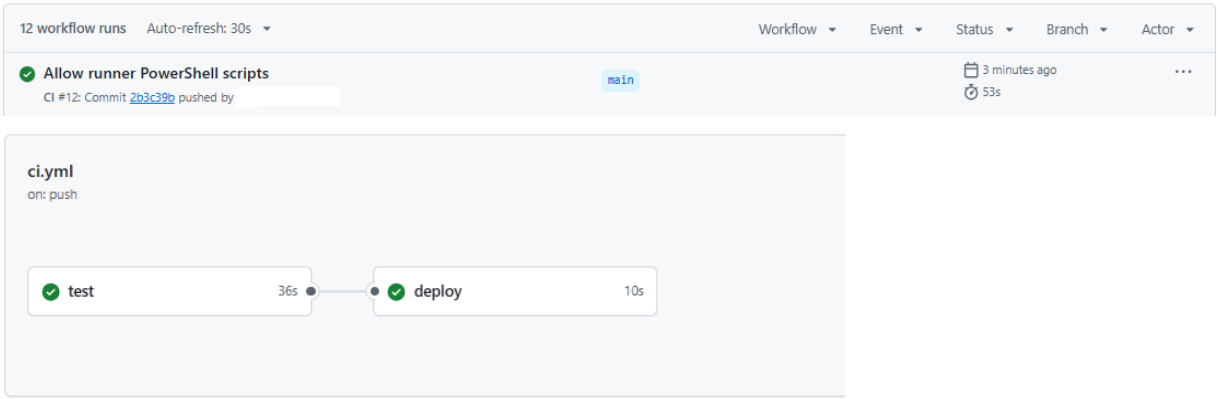


File explanation:

- **needs: test** makes this job wait until the existing CI job succeeds.
- **runs-on** selects your self-hosted runner with the portfolio-deploy label. GitHub documentation
- **if and shell** choose the appropriate command shell for Windows or macOS.

- **docker info** verifies that the runner can communicate with Docker Desktop. We'll add the actual deployment commands after this check succeeds.

3. Commit your changes and push them to github, the action should run successfully for both "test" and "deploy"



Part 7: Download the published image automatically

1. Add the below snippet at the end of the ci.yml file, always check for correct indentations. (Replace YOUR_GITHUB_NAME_HERE)

```
- name: Log in to GitHub Container Registry
  uses: docker/login-action@v4
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}

- name: Pull image on Windows
  if: runner.os == 'Windows'
  shell: powershell -NoProfile -ExecutionPolicy Bypass -File {0}
  run: docker pull --platform linux/amd64 ghcr.io/ali-moukallid/portfolio_jenkins:latest

- name: Pull image on macOS
  if: runner.os == 'macOS'
  shell: bash
  run: docker pull --platform linux/amd64 ghcr.io/ali-moukallid/portfolio_jenkins:latest
```

```yaml
deploy:
  needs: test
  runs-on: [self-hosted, portfolio-deploy]
  steps:
    - name: Check Docker on Windows
      if: runner.os == 'Windows'
      shell: powershell -NoProfile -ExecutionPolicy Bypass -File {0}
      run: docker info

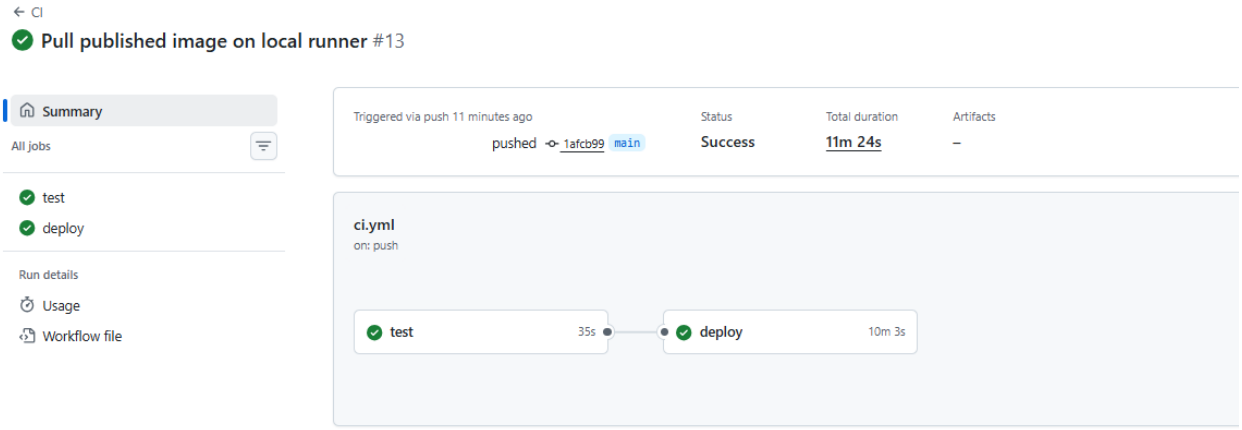
    - name: Check Docker on macOS
      if: runner.os == 'macOS'
      shell: bash
      run: docker info

    - name: Log in to GitHub Container Registry
      uses: docker/login-action@v4
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}

    - name: Pull image on Windows
      if: runner.os == 'Windows'
      shell: powershell -NoProfile -ExecutionPolicy Bypass -File {0}
      run: docker pull --platform linux/amd64 ghcr.io/[redacted]/portfolio_jenkins:latest

    - name: Pull image on macOS
      if: runner.os == 'macOS'
      shell: bash
      run: docker pull --platform linux/amd64 ghcr.io/[redacted]/portfolio_jenkins:latest
```

2. Commit your changes and push them to github, the action should run successfully for both “test” and “deploy”



Part 8: Start or replace the app container automatically

1. Add the below snippet at the end of the ci.yml file, always check for correct indentations. (Replace YOUR_GITHUB_NAME_HERE)

```
      - name: Deploy on Windows
        if: runner.os == 'Windows'
        shell: powershell -NoProfile -ExecutionPolicy Bypass -File {0}
        run: |
          $existing = docker ps -aq --filter "name=^/portfolio-app$"
          if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }

          if ($existing) {
            docker rm -f portfolio-app
            if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }
          }

          docker run -d --platform linux/amd64 --name portfolio-app -p 127.0.0.1:8000:8000 ghcr.io/ali-moukallid/portfolio_jenkins:latest
          exit $LASTEXITCODE

      - name: Deploy on macOS
        if: runner.os == 'macOS'
        shell: bash
        run: |
          existing=$(docker ps -aq --filter "name=^/portfolio-app$")
          if [ -n "$existing" ]; then
            docker rm -f portfolio-app
          fi

          docker run -d --platform linux/amd64 --name portfolio-app -p 127.0.0.1:8000:8000 ghcr.io/ ali-moukallid/portfolio_jenkins:latest
```

```yaml
deploy:
  steps:
    - name: Log in to GitHub Container Registry
      with:
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}

    - name: Pull image on Windows
      if: runner.os == 'Windows'
      shell: powershell -NoProfile -ExecutionPolicy Bypass -File {0}
      run: docker pull --platform linux/amd64 ghcr.io/[redacted]/portfolio_jenkins:latest

    - name: Pull image on macOS
      if: runner.os == 'macOS'
      shell: bash
      run: docker pull --platform linux/amd64 ghcr.io/[redacted]/portfolio_jenkins:latest

    - name: Deploy on Windows
      if: runner.os == 'Windows'
      shell: powershell -NoProfile -ExecutionPolicy Bypass -File {0}
      run: |
        $existing = docker ps -aq --filter "name=^/portfolio-app$"
        if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }

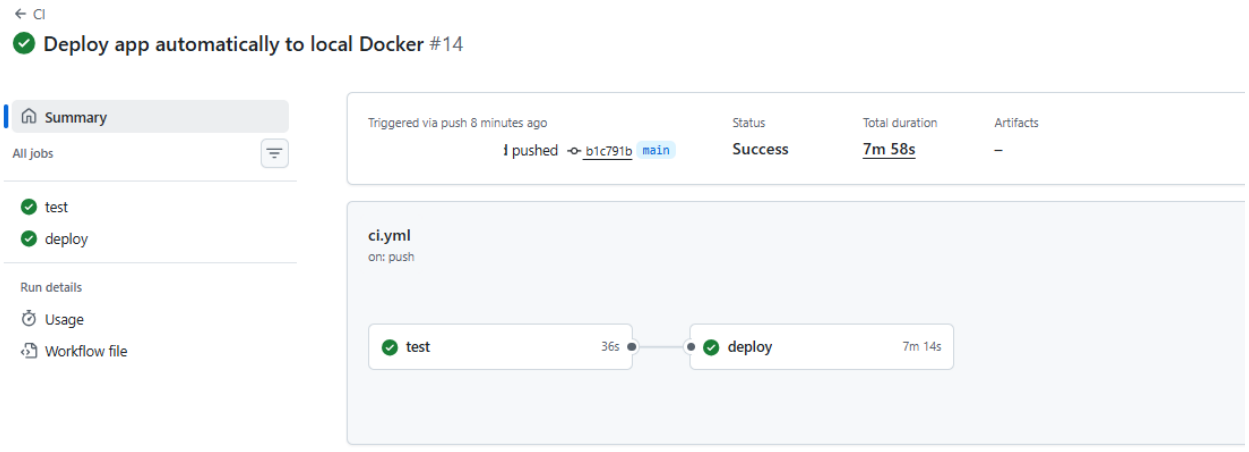
        if ($existing) {
          docker rm -f portfolio-app
          if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }
        }

        docker run -d --platform linux/amd64 --name portfolio-app -p 127.0.0.1:8000:8000 ghcr.io/[redacted]/portfolio_jenkins:latest
        exit $LASTEXITCODE

    - name: Deploy on macOS
      if: runner.os == 'macOS'
      shell: bash
      run: |
        existing=$(docker ps -aq --filter "name=^/portfolio-app$")
        if [ -n "$existing" ]; then
          docker rm -f portfolio-app
        fi

        docker run -d --platform linux/amd64 --name portfolio-app -p 127.0.0.1:8000:8000 ghcr.io/[redacted]/portfolio_jenkins:latest
```

2. Commit your changes and push them to github, the action should run successfully for both “test” and “deploy”



Go to localhost:8000, the portfolio should appear

Part 9: Testing

To test, make any visible change to your UI (for example in templates>home.html), wait for the action to run successfully and refresh localhost:8000. The change should apply automatically.